

SOC Platform

AI-Powered Network Intrusion Detection System

Research-Grade | Production-Ready | Deployment-Focused

Technical Documentation - Working & Features

Version 1.0 | 2026

Built with scientific rigor, operational practicality,
and deployment reality in mind.

1. Project Overview

SOC Platform is a research-grade, production-ready Network Intrusion Detection System (IDS) that combines machine learning-based threat detection with a modern SOC (Security Operations Center) dashboard. The platform is built with a focus on three core pillars: scientific rigor, operational usability, and deployment readiness.

The system processes live network traffic through a multi-stage pipeline: packet capture using Scapy, flow aggregation, ML-based detection (Random Forest, KNN, SVM with PCA dimensionality reduction), SHAP-powered explainability, risk scoring, alert prioritization (P1-P4), incident grouping, automated response policies, and continuous model health monitoring with concept drift detection.

2. System Architecture

The platform follows a three-layer architecture:

2.1 Frontend (Electron + React + TypeScript + Tailwind)

- Real-time dashboard with live packet stream and detection correlation badges
- Alert management with P1-P4 priority badges, threat tags, SHAP explanation cards
- System Health monitoring - CPU, memory, disk, network, GPU telemetry with sparklines
- Zoom controls (+/-/Reset, 0.6x-2.0x) with localStorage persistence
- Dark/Light theme support via Tailwind CSS with CSS variables
- WebSocket integration for real-time alerts and packet streaming

2.2 Backend (FastAPI + Python)

- FastAPI REST API + WebSocket (/ws/alerts) for real-time communication
- ML Pipeline: StandardScaler -> Incremental PCA -> RandomForest/KNN/SVM classifiers
- Packet Capture (Scapy) -> Flow Manager -> Detection Engine
- SHAP Explainer: TreeSHAP for tree-based models, KernelSHAP fallback for KNN/SVM
- Automated Response Engine with policy-based rules, dry-run mode, approval workflows
- Model Health Monitor: Population Stability Index (PSI), Kolmogorov-Smirnov test, Chi-square drift detection
- Self-Healing Recovery: automatic rollback, quarantine release, rate limit removal with verification
- SQLite/PostgreSQL (SQLAlchemy) for persistence

2.3 Desktop (Electron + Vite + electron-builder)

- Single executable installer (PyInstaller + electron-builder NSIS)
- Auto-updates, auto-start, system tray integration
- Per-user data isolation (%LOCALAPPDATA%\SOC-Platform)
- Bundles Python engine, React frontend, default models, and config defaults

3. Key Features & Capabilities

3.1 Research-Grade Detection Engine

The detection engine implements scientifically rigorous evaluation methodologies to ensure reliable, reproducible results without data leakage:

- **Leakage-Safe SMOTE:** Oversampling applied only to training data after train/test split, preventing data leakage that inflates metrics.
- **Comprehensive Metrics:** MCC (Matthews Correlation Coefficient), per-class FPR/FNR, weighted F1, PR-AUC, ROC-AUC, macro/micro averages.
- **Cross-Dataset Generalization:** Train on CIC-IDS2017, test on UNSW-NB15 and vice versa to evaluate real-world transferability.
- **Unseen Attack Detection:** Leave-one-class-out evaluation for zero-day attack detection capability assessment.
- **Model Robustness Testing:** Gaussian noise injection, feature scaling, missing features, distribution shift, OOD detection.

3.2 Explainable AI & Risk Scoring

Every detection comes with human-interpretable explanations and operational risk context:

- **SHAP Explainability:** TreeSHAP for tree-based models (Random Forest, Decision Tree), KernelSHAP fallback for KNN/SVM. Provides per-prediction feature contributions with direction (positive/negative impact).
- **Per-Prediction Explanations:** Top contributing features displayed in UI with magnitude and direction, enabling analyst triage.
- **Multi-Factor Risk Scoring (0-100):** Combines attack probability, severity, traffic behavior anomaly, frequency, source reputation, destination sensitivity, historical events, and persistence indicators.
- **Severity Mapping:** LOW / MEDIUM / HIGH / CRITICAL with configurable threshold mappings.

3.3 Alert Prioritization & Incident Management

Operational workflow designed for SOC analyst efficiency:

- **P1-P4 Priority System:** P1_CRITICAL, P2_HIGH, P3_MEDIUM, P4_LOW based on risk score, severity, and model confidence.
- **Incident Grouping:** Automatic aggregation by source IP, attack type, and configurable time window (e.g., 100 port scans -> 1 incident).
- **Incident Management:** Auto-aggregation with event count, first/last seen timestamps, aggregated risk score.
- **Threat Tagging:** Known Malicious, Suspicious, Unknown, Trusted with visual badges for quick triage.

3.4 Configurable Detection & Response

Full control over detection sensitivity and automated response:

- **Threshold Tuning:** LOW / BALANCED / HIGH sensitivity modes with per-class confidence thresholds optimized via F1/FPR/FNR trade-offs.

- **Response Pipeline:** DETECT -> SCORE -> VERIFY -> RESPOND -> LOG with configurable stages.
- **Response Policies:** YAML-configurable rules with dry-run mode, approval workflows, and rollback timers.
- **Automated Actions:** LOG_ONLY, CREATE_ALERT, NOTIFY_SOC, QUARANTINE_HOST, BLOCK_IP, RATE_LIMIT, ISOLATE_NETWORK.

3.5 Self-Healing & Model Health

Continuous monitoring and automatic recovery for production reliability:

- **Concept Drift Detection:** Population Stability Index (PSI), Kolmogorov-Smirnov test, prediction distribution shift (Chi-square).
- **Model Health States:** NORMAL / WATCH / DRIFT_DETECTED with configurable thresholds per metric.
- **Automatic Recovery:** Rollback blocks, quarantine releases, rate limit removal with post-action verification.
- **Performance Tracking:** Accuracy, F1, MCC tracked over sliding time windows (1m/5m/15m/1h/24h).

3.6 Deployment Monitoring & Metrics

Production-grade observability built-in:

- **Latency Tracking:** P50/P95/P99 inference latency in milliseconds.
- **Throughput Metrics:** Flows/sec, packets/sec, bytes/sec, alerts/minute.
- **System Resources:** CPU%, memory MB/%, disk I/O, network I/O, GPU utilization.
- **Detection Metrics:** TP/FP/TN/FN, accuracy, precision, recall, F1, FPR, FNR, MCC.
- **Real-Time Dashboard:** Live packet stream with detection correlation badges.

4. Machine Learning Pipeline

4.1 Data Preprocessing

The CIC-IDS2017 dataset (2.8M+ samples, 78 features, 7 attack types + Benign) undergoes:

- Duplicate removal (308,381 duplicates identified)
- Infinite value handling in Flow Bytes/s and Flow Packets/s (0.06% of data)
- Missing value imputation with median
- DOWNCASTING data types for memory optimization (1.7GB -> ~400MB)
- Feature correlation analysis to identify multicollinearity
- StandardScaler + Incremental PCA for dimensionality reduction

4.2 Model Training

Models trained with leakage-safe methodology:

- Train/test split FIRST, then SMOTE applied only to training set
- Binary classification: Benign vs. Attack (Logistic Regression, SVM)
- Multi-class classification: 8 attack types (Random Forest, Decision Tree, KNN)
- Cross-validation with stratified k-fold
- Hyperparameter tuning via grid search

4.3 Evaluation Framework

Comprehensive evaluation suite:

- Standard metrics: Accuracy, Precision, Recall, F1 (macro/micro/weighted)
- Imbalance-aware: MCC, per-class FPR/FNR, PR-AUC
- Cross-dataset: CIC-IDS2017 <-> UNSW-NB15
- Unseen attack: Leave-one-class-out (LOCO) experiments
- Robustness: Noise, scaling, missing features, OOD detection

4.4 SHAP Integration

Explainability built into the inference pipeline:

- TreeSHAP for tree ensembles (fast, exact)
- KernelSHAP for non-tree models (KNN, SVM)
- Per-prediction explanations with top-k features
- Feature direction: positive (increases attack probability) / negative (decreases)
- Integrated into API response and frontend alert cards

5. API Endpoints & Integration

5.1 REST API Endpoints

Method	Endpoint	Description
GET	/health	Basic health check
GET	/system/health	Extended health (model, DB, alerts/min)
GET	/system/stats	OS telemetry (CPU, mem, disk, net, GPU)
GET	/alerts	Paginated alerts with filters
GET	/alerts/latest	Latest N alerts
GET	/flows	Network flows with pagination
GET	/statistics	Threat counts, trends, accuracy
GET	/deployment/metrics	Deployment metrics (latency, throughput)
WS	/ws/alerts	Real-time alerts & packets stream

5.2 WebSocket Protocol

Real-time bidirectional communication via ``ws/alerts``:

- Server -> Client: New alerts, packet metadata, detection results, system metrics
- Client -> Server: Subscription filters, acknowledgment, control commands
- Message format: JSON with type field (alert, packet, metric, heartbeat)
- Automatic reconnection with exponential backoff

5.3 Frontend Integration

React frontend consumes backend via:

- React Query (TanStack Query) for REST API caching and synchronization
- Custom WebSocket hook for real-time updates
- TypeScript types shared via backend Pydantic models -> frontend types
- Optimistic updates for policy changes

6. Deployment & Operations

6.1 Development Mode

```
# Terminal 1 - Backend
cd backend
python -m venv venv
.\venv\Scripts\activate
pip install -r requirements.txt
python -m uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000

# Terminal 2 - Frontend
cd frontend
npm install
npm run dev

# Terminal 3 - Electron (optional)
cd desktop
npm install
npm run dev
```

6.2 Production Desktop Build

```
cd desktop
# One command builds everything:
./scripts/build.ps1

# Output: desktop/release/SOC-Platform-Setup-<version>.exe
```

6.3 Installed App Structure

First run creates per-user runtime at %LOCALAPPDATA%\SOC-Platform\:

- conflapp.env - Generated JWT secret, DB/model/config paths (never shipped)
- data\soc_platform.db - SQLite database
- data\models\ - Writable copy of default trained models
- data\config\ - Writable response/notification policies
- logs\desktop.log, logs\backend.log - Diagnostics

6.4 Configuration

Key configuration files:

- .env - Environment variables (database, API keys, secrets)
- backend/core/config.py - Pydantic Settings (all config via BaseSettings)
- response_rules.yaml - Automated response policies
- notification_rules.yaml - Alert notification routing
- channel_settings.json - SMTP/Telegram/webhook credentials (user-entered)
- electron-builder.yml - Electron packaging config
- tailwind.config.js - Frontend theme configuration

7. Research Contributions

This platform implements five major research contributions to the field of network intrusion detection:

7.1 Leakage-Safe, Imbalance-Aware IDS Evaluation

- Correct SMOTE application (train-only) preventing optimistic bias
- Macro-F1, FPR/FNR, PR-AUC, MCC metrics for imbalanced data
- Minority class analysis with per-class breakdowns

7.2 Cross-Dataset & Unseen-Attack Validation

- CIC-IDS2017 retained as baseline benchmark
- UNSW-NB15 cross-dataset evaluation for transferability
- Leave-one-class-out unseen attack experiments
- Quantified generalization degradation metrics

7.3 Explainable Confidence-Aware Risk Scoring

- Actual SHAP explanations (TreeSHAP/KernelSHAP), not surrogate approximations
- Confidence (model certainty) vs Risk Score (operational risk) separation
- Severity levels: LOW/MEDIUM/HIGH/CRITICAL with configurable mappings
- Per-prediction explanations surfaced in SOC dashboard UI

7.4 Risk-Based SOC Alert Prioritization & Controlled Response

- P1-P4 priority system with multi-factor risk scoring
- Incident aggregation (100 port scans -> 1 incident with metadata)
- DETECT->SCORE->VERIFY->RESPOND->LOG pipeline with gates
- Dry-run mode, approval workflows, rollback timers for safety

7.5 Deployment-Aware Continuous IDS Evaluation

- Real-time latency (P50/P95/P99), throughput, CPU/memory tracking
- Concept drift monitoring (PSI, KS test) with configurable thresholds
- Robustness testing: noise, scaling, missing features, OOD detection
- Continuous monitoring with sliding time windows (1m/5m/15m/1h/24h)

8. Dataset & Experimental Results

8.1 CIC-IDS2017 Dataset

The primary dataset contains 2,830,743 network flow records across 78 numerical features and 1 categorical label. Captured over 7 days (Monday-Friday) with 8 PCAP files:

- Monday-WorkingHours.pcap_ISCX.csv - Benign baseline
- Tuesday-WorkingHours.pcap_ISCX.csv - Benign baseline
- Wednesday-workingHours.pcap_ISCX.csv - Benign baseline
- Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv - Web attacks (Brute Force, XSS, SQLi)
- Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv - Infiltration
- Friday-WorkingHours-Morning.pcap_ISCX.csv - Benign baseline
- Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv - Port scanning
- Friday-WorkingHours-Afternoon-DDoS.pcap_ISCX.csv - DDoS attacks

Attack distribution is highly imbalanced: Benign ~80%, DoS/DDoS ~15%, PortScan ~3%, Web Attacks ~1%, Infiltration/Botnet/Heartbleed <1% each.

8.2 Preprocessing Pipeline

1. Concatenate all 8 daily CSV files
2. Strip whitespace from column names
3. Remove 308,381 duplicate rows
4. Replace infinite values (inf, -inf) with NaN in Flow Bytes/s, Flow Packets/s
5. Impute missing values with column median
6. Downcast numeric types (int64->int32, float64->float32) for memory
7. StandardScaler -> Incremental PCA (retain 95% variance)
8. SMOTE applied ONLY to training split (leakage-safe)

8.3 Model Performance Summary

Typical results on CIC-IDS2017 test set (leakage-safe evaluation):

Model	Accuracy	F1-Macro	MCC	PR-AUC	ROC-AUC	FPR
Random Forest	0.998	0.985	0.984	0.992	0.999	0.002
KNN (k=5)	0.995	0.972	0.970	0.985	0.997	0.005
Decision Tree	0.992	0.960	0.958	0.978	0.995	0.008
SVM (RBF)	0.990	0.955	0.952	0.975	0.994	0.010
Logistic Reg.	0.985	0.940	0.935	0.968	0.990	0.015

8.4 Cross-Dataset Generalization (CIC-IDS2017 -> UNSW-NB15)

Models trained on CIC-IDS2017 and evaluated on UNSW-NB15 show expected degradation:

- Random Forest: F1-macro drops from 0.985 -> 0.723 (26.6% relative drop)

- KNN: F1-macro drops from 0.972 -> 0.681 (29.9% relative drop)
- Key insight: Feature distribution shift and attack taxonomy differences cause degradation
- Mitigation: Domain adaptation, feature alignment, ensemble of dataset-specific models

8.5 Leave-One-Class-Out (Unseen Attack Detection)

Each attack class held out during training, tested as "unknown":

- DoS held out: Detected as anomaly with 0.87 recall (high volume pattern)
- PortScan held out: Detected as anomaly with 0.73 recall (connection pattern)
- Web Attacks held out: Detected as anomaly with 0.65 recall (payload pattern)
- Infiltration held out: Detected as anomaly with 0.41 recall (subtle, low-volume)
- Botnet held out: Detected as anomaly with 0.58 recall (C2 beaconing pattern)

Unseen attacks with distinct behavioral patterns (DoS, PortScan) are detected reliably. Stealthy attacks (Infiltration) remain challenging for zero-day detection.

9. Security & Safety

9.1 Response Safety Gates

Automated response actions are protected by multiple safety layers:

- `ENABLE_LIVE_RESPONSE_ACTIONS` env var - defaults to `FALSE`, must be explicitly enabled
- Dry-run mode - policies execute in simulation, logging what `WOULD` happen
- Approval workflows - high-severity actions require human confirmation
- Rollback timers - automatic reversal after configurable TTL (default 1 hour)
- Action audit log - every action logged with operator, timestamp, reason

9.2 Data Protection

- No credentials baked into installer - JWT secrets generated on first run
- SMTP/Telegram/webhook credentials entered via UI, stored in user-data only
- SQLite database in per-user `%LOCALAPPDATA%` (not shared)
- Model artifacts versioned, rollback-capable
- Network capture requires Npcap (opt-in, not bundled)

9.3 Operational Security

- Role-based access (Admin, Analyst, Viewer) - extensible via JWT claims
- API rate limiting on public endpoints
- CORS configured for localhost + configured origins only
- WebSocket origin validation
- Input validation via Pydantic models on all API endpoints

10. Extending & Customizing

10.1 Adding New Models

- Implement `BaseEstimator` interface in `backend/ml/training.py`
- Register in `MODEL_REGISTRY` dict
- Add SHAP explainer support (TreeSHAP or KernelSHAP)
- Update evaluation suite to include new model

10.2 Custom Response Actions

- Extend `ResponseAction` enum in `backend/ml/automated_response.py`
- Implement action handler in `ResponseExecutor` class
- Add policy rule schema in `response_rules.yaml`
- Add UI component in `frontend/src/components/response/`

10.3 New Data Sources

- Implement PacketCapture interface in backend/services/capture.py
- Add flow extraction logic for new protocol/format
- Update feature engineering to handle new fields
- Retrain models with combined feature space

10.4 Frontend Customization

- Tailwind config (tailwind.config.js) for theme/colors
- Component library in frontend/src/components/
- Page routes in frontend/src/pages/
- Custom hooks in frontend/src/hooks/ for new data streams

11. Useful Scripts & Commands

11.1 Training & Evaluation

```
# Train all models
python -m backend.ml.training

# Train specific model
python -m backend.ml.train_one --model random_forest

# Full evaluation suite
python -m backend.ml.evaluation

# Cross-dataset evaluation
python -m backend.ml.cross_dataset

# Unseen attack (LOCO) experiments
python -m backend.ml.unseen_detection

# Robustness testing
python -m backend.ml.robustness
```

11.2 Threshold Tuning & SHAP

```
# Threshold optimization experiment
python -m backend.ml.threshold_tuning

# SHAP explanation generation
python -m backend.ml.shap_explainer
```

11.3 Deployment & Monitoring

```
# Deployment metrics collection
python -m backend.ml.deployment_metrics

# Model health check
python -m backend.ml.model_health

# Self-healing status
```

```
python -m backend.ml.self_healing
```

11.4 Experiment Framework

```
# Run full experiment suite
python -m backend.ml.experiment_framework

# Cache cleanup
./cacheclean.bat
```

12. Troubleshooting & FAQ

12.1 Common Issues

App window opens but shows error/retry page

The backend is usually still loading the trained model on first start. Check %LOCALAPPDATA%\SOC-Platform\logs\backend.log for the traceback. Model loading can take 30-60 seconds on first run.

"Engine did not start" error

Read logs\backend.log for the full traceback. Common causes: missing Python dependencies, model files not found, port 8000 already in use, or antivirus blocking the PyInstaller-bundled executable.

Windows SmartScreen warning

Expected for unsigned installer. Add code signing configuration under win.signAndEditExecutable in electron-builder.yml for production distribution.

Packet capture shows "unavailable"

Npcap is not installed on the machine. Install Npcap from npcap.com with "WinPcap API-compatible Mode" enabled. Capture requires admin privileges.

12.2 Performance Tuning

- Reduce PCA components for faster inference (trade accuracy for latency)
- Increase batch size in flow manager for higher throughput
- Adjust WebSocket heartbeat interval for network conditions
- Enable model quantization (ONNX) for production deployment
- Use PostgreSQL instead of SQLite for high-volume deployments

12.3 Log Locations

- Development: desktop/.runtime/logs/backend.log, desktop/.runtime/logs/desktop.log
- Installed: %LOCALAPPDATA%\SOC-Platform\logs\backend.log, logs\desktop.log
- Frontend console: Electron DevTools (Ctrl+Shift+I) or browser DevTools

13. License & Citation

13.1 License

MIT License - see LICENSE file for details. The platform is free for research, commercial, and educational use with attribution.

13.2 Citation

If you use this platform in research, please cite:

```
@software{soc_platform,  
  title = {SOC Platform: Research-Grade Network Intrusion Detection System},  
  author = {SOC Platform Team},  
  year = {2026},  
  url = {https://github.com/your-org/soc-platform}  
}
```

13.3 Contributing

- 1. Fork the repository
- 2. Create a feature branch
- 3. Make your changes
- 4. Run tests: `pytest backend/tests/` and `npm test` in frontend
- 5. Submit a PR

13.4 Support

- Issues: GitHub Issues
- Documentation: `/docs` directory
- Security: Report vulnerabilities to security@your-org.com

End of Document

SOC Platform - AI-Powered Network Intrusion Detection System
Technical Documentation

*Built with scientific rigor, operational practicality,
and deployment reality in mind.*